

TKOM 2026L - Dokumentacja wstępna, Mateusz Jakubiak

Założenia projektowanego języka

- Statycznie typowany - typ zmiennej ustalony przy deklaracji nie może na późniejszym etapie ulec zmianie..
- Silnie typowany, tj. konwersje muszą być jawne.
- Zmienne domyślnie niemutowalne.
- Wbudowany typ kolekcji homogenicznych, `list<T>`
- Możliwość tworzenia typów użytkownika poprzez `class`.
- Głęboka niemutowalność dla obiektów klas użytkownika
- Przekazywanie argumentów funkcji przez wartość

Typy wbudowane w języku, statyczne typowanie, konwersje

```
int
uint
float
bool
char
string
list<T>
```

Typy wyżej wymienione są typami podstawowymi (prostymi) projektowanego języka. Podczas analizy semantycznej są one przypisywane danym zmiennym, i nie jest możliwa późniejsza zmiana typu zmiennej, zgodnie z ideą statycznego typowania.

Jeśli nie definiujemy zmiennych, a jedynie je deklarujemy, przyjmują one domyślną wartość:

0 dla `int/uint/char`

0.0 dla `float`

false dla `bool`

"" dla `string`

[] dla `list`

Zapisanie konwersji w poniższych punktach jako **typ as typ** (przykładowo, **int as float**) jest skrótem myślowym, faktycznie konwersji dokonuje się przez użycie nazw zmiennych po lewej stronie operatora `as`

```
int x = 3
float x_float = x as float
```

1. Typ `int`:

Zakres wartości wynosi od -2 147 483 648 do 2 147 483 647.

Możliwe konwersje do innych typów:

- **int as bool** - dla dodatnich wartości zmiennej typu int konwersja skutkuje otrzymaniem true, w przeciwnym przypadku false
- **int as uint** - Jeżeli wartość int mieści się w zakresie typu uint (0 do 4 294 967 295), konwersja jest poprawna. W przypadku wartości spoza tego zakresu, interpreter zgłasza błąd konwersji, jednak kontynuuje dalsze wykonywanie programu .djm i dokonuje operacji zawinięcia na konwertowanej zmiennej typu int.
- **int as float**
- **int as string**
- **int as char** - W przypadku gdy int wykracza poza zakres char (0-255), używany jest mechanizm zawinięcia i kontynuowane jest interpretowanie programu, a interpreter zgłasza błąd użytkownikowi.

2. Typ uint:

Zakres wartości wynosi od 0 do 4 294 967 295.

- **uint as bool** - konwersja zwraca false gdy zmienna typu uint ma wartość 0, true w przeciwnym przypadku
- **uint as int** - Jeśli wartość zmiennej typu uint mieści się w zakresie typu int (do 2 147 483 647), konwersja przebiega poprawnie. W przeciwnym przypadku zgłaszany jest błąd, ale interpreter używa operacji modulo i kontynuuje wykonanie programu.
- **uint as float**
- **uint as string**
- **uint as char** - W przypadku gdy int wykracza poza zakres char (0-255), używany jest mechanizm zawinięcia, a interpreter zgłasza błąd użytkownikowi.

3. Typ float:

Typ float reprezentuje liczbę zmiennoprzecinkową podwójnej precyzji.

- **float as bool**
- **float as int** - W przypadku gdy po odrzuceniu części po przecinku zmiennej typu float, otrzymana wartość całkowita wykracza poza zakres int, używany jest mechanizm zawinięcia za pomocą operacji modulo, a interpreter zgłasza błąd.
- **float as uint** - W przypadku gdy po odrzuceniu części po przecinku zmiennej typu float, otrzymana wartość całkowita wykracza poza zakres uint, używany jest mechanizm zawinięcia za pomocą operacji modulo, a interpreter zgłasza błąd.
- **float as string**
- **float as char** - Odpowiada konwersji (float as int) as char

4. Typ bool

Reprezentuje wartość prawda/fałsz. Domyślnie zwracany jest przez operatory równości, nierówności, mniejszości, większości. Definiowanie zmiennej tego typu może być dokonane przez użycie słów kluczowych języka **true** lub **false**.

```
bool b = true
bool b1 = false
bool b2 = 2 == 3
```

- **bool as int**
- **bool as uint**
- **bool as float**
- **bool as char**

5. Typ char

Reprezentuje pojedynczy znak zakodowany w 8-bitowym ASCII (czyli ASCII rozszerzonym). Literały char są zapisywane pomiędzy apostrofami ('a'). Możliwe są operacje arytmetyczne dokonywane na zmiennych typu char. W przypadku przepełnienia jego zakresu (0-255), interpreter ostrzega użytkownika w sposób analogiczny do przypadku przepełnienia zmiennych typu int/uint.

W ogólności można więc traktować typ char jako typ liczbowy, przechowujący liczby z zakresu 0-255 (czyli używający do zapisu jednego bajta).

- **char as int** - konwertuje char na liczbę z zakresu 0-255
- **char as uint**
- **char as float** - odpowiada (char as int) as float
- **char as bool** - zwraca true, gdy char jest większe od 0 (co w tablicy znaków ASCII jest przedstawiane jako NULL)
- **char as string**

6. Typ string

Typ string jest semantycznie traktowany jako list<char> zakończony 0 - wprowadzenie takiego podejścia do tego typu do standardu projektowanego języka .djm upraszcza implementację jego interpretera.

- **string as int** - możliwe tylko, gdy string składa się z samych liczb i separatorów (podkreślników) oraz ewentualnego znaku minus na początku. Próba użycia tej konwersji na zmiennej typu string nie spełniającej powyższych warunków skutkuje błędem semantycznym. W przypadku wyjścia poza zakres int liczby zapisanej w zmiennej string dochodzi do operacji zawinięcia (z użyciem modulo), a interpreter .djm zgłasza błąd.
- **string as uint** - konwersja działająca w sposób analogiczny do **string as int**
- **string as float** - konwersja działająca w sposób analogiczny do **string as int**, dopuszczająca jednak także kropkę oddzielającą wartość całkowitą od wartości ułamkowej.

- Operator konwersji **as** nie jest zdefiniowany dla przypadku **string as char**, jednak ze względu na sposób przedstawienia string, wystarczy użyć operatora indeksowania listy `[]`.

7. Typ `list<T>`

Typ `list<T>` jest uporządkowaną kolekcją przechowującą elementy jednego typu `T` (a zatem homogeniczną). Jest uporządkowana (kolejność ma znaczenie), oraz może zawierać duplikaty elementów. Dopuszczalne jest tworzenie list zagnieżdżonych typu `list<list<T>>`.

Dla listy jest zdefiniowany operator `!=` oraz `==`, dokonujący porównania wszystkich elementów obydwu list po kolei. Operator `==` zwraca fałsz, wraz z wypisaniem ostrzeżenia dla użytkownika interpretera, w przypadku gdy porównywane listy mają różną długość.

Możliwe jest użycie operatora konwersji **as** dla list.

```
list<float> xs = [1.1, -2.5, 3.2]
list<int> ys = xs as list<int>
```

Po takiej konwersji, do `ys` zostanie przypisana wartość `[1, -2, 3]`.

Konwersja elementów listy podlega takim samym zasadom i restrykcjom, jak konwersja poszczególnych typów prostych, wypisanych w poprzedniej części tego dokumentu.

Literały

Język dopuszcza literały: liczb całkowitych, liczb zmiennoprzecinkowych, logiczne, typu `char`, typu `string` oraz literały list (w tym list zagnieżdżonych).

Literały liczby całkowitych mogą być poprzedzone minusem oraz zawierać separatory w postaci podkreślników. Literały całkowitoliczbowe dodatnie mogą być dopasowane przez analizator semantyczny, w zależności od kontekstu, do typu `int`, `uint` lub `float`. Jest to jedyny w języku mechanizm mogący być uznany za przejaw słabego typowania. Ze względu na obecność tego mechanizmu, wszystkie spośród poniższych wyrażeń są w języku traktowane jako w pełni poprawne:

```
int i = 10
uint u = 16
float f = 2.5
float f1 = f / 5      // Równoznaczne f / 5.0
float f2 = 3          // Równoznaczne float f2 = 3.0
```

Ten mechanizm dopasowania literału dodatniego całkowitoliczbowego nie jest obecny dla typów różnych od `int/uint/float`:

```
char c = 'a'
if c == 2 { }
```

Error [Semantic] at line X, column Y, during operation: c == 2
Operator == not defined for [char] == [IntegerLiteral]

```
char c = 5
```

Error [Semantic] at line X, column Y, during assignment: char c = 5
Couldn't assign IntegerLiteral to variable of type char

W przypadku, gdy zapisany literał całkowitoliczbowy wykracza poza zakres typu do którego analizator semantyczny zdecyduje się owy literał przypisać, interpreter zgłasza błąd i za pomocą operacji modulo dokonuje zawinięcia i kontynuuje próbę wykonania programu.

```
int x = 3'000'000'000
```

Error [Semantic] at line X, column Y, during assignment: int x = 5'000'000'000"
Provided integer literal does not fit into type int

```
uint u = 3'000'000'000 // Poprawne
```

Literały liczb z częścią dziesiętną zawierają w sobie kropkę, rozdzielającą ją od części całkowitej i traktowane są jako typ float. Jeśli dany literał nie jest możliwy do zapisania liczbą zmiennoprzecinkową o podwójnej precyzji (ze względu na brak wystarczającej precyzji), interpreter zgłosi ostrzeżenie (Warning). W literałach liczb z częścią dziesiętną także możliwe jest użycie separatorów w postaci podkreślników.

Literały typu char są znakiem zapisanym między apostrofami, a **literały typu string** pomiędzy cudzysłowami. Różnicę obrazuje konieczność użycia operatora konwersji w poniższym przykładzie:

```
mut string s = "a"  
s = s + 'b' as char  
s == "ab" // true
```

Literały list tworzone są przez zapisanie ich elementów pomiędzy nawiasami kwadratowymi []. Operatory list, opisane w późniejszej części tego dokumentu, mogą działać zarówno na literałach list, jak i na nazwach zmiennych typu listowego. Niech zobrazują to poniższy przykłady:

```
list<int> listOfInts = [2, 5]  
list<int> listOfInts2 = listOfInts + [7]  
listOfInts contains [2] // true
```

```
for list<int> i in [ [1,2], [3,4], [5, 6] ] {  
    // zmienna i będzie wynosić po kolei: [1,2], potem [3,4], potem [5,6]  
}
```

Literały listowe umożliwiają zapisywanie także list zagnieżdżonych. Należy nadmienić, że listy są w języku .djm homogeniczne, i utworzenie literału złożonego z typów mieszanych skutkuje błędem na etapie analizy semantycznej.

```
[1, 2, "a", 3]
```

Error [Semantic] at line X, column Y, during semantic analysis: [1, 2, "a", 3]

Mixed type list is not supported

Co do zasady, literał listy pustej [] nie jest określony jako zawierający konkretny typ, i po przekazaniu analizatorowi semantycznemu przez parser jako EmptyListLiteral (nazwa tymczasowa) może zostać przez owy analizator dopasowany do konkretnego typu listowego, w tym zagnieżdżonego.

Operator przypisania

W ogólnym ujęciu, w języku użycie operatora przypisania wartości do zmiennej

```
T a = expr
```

Skutkuje utworzeniem kopii (w przypadku list zagnieżdżonych i obiektów klas - głębokiej) wyrażenia po prawej stronie przypisania, zgodnie z poniższym przykładem:

```
list<int> xs = [1, 2, 3]
```

```
list<int> ys = xs
```

```
mut ys = ys
```

```
ys = ys + [4]
```

```
xs == [1, 2, 3]           // true
```

```
ys == [1, 2, 3, 4]       // true
```

Operatory matematyczne

W języku .djm dostępne do użycia są podstawowe operatory matematyczne:

dodawania (+)

odejmowania (-)

mnożenia (*)

dzielenia (/)

potęgowania (^)

Ich priorytety zapewniają kolejność działań zgodną z tą znaną z podstaw matematyki.

Wszystkie z nich są zdefiniowane dla par zmiennych typów: int, uint, float, char, przy czym char jest semantycznie traktowany jako liczba z zakresu 0-255. Na skutek ich działania otrzymujemy wynik o typie zgodnym z tym posiadanym przez zmienne (literały) na których ich użyto.

int	+ - * / ^	int	-> int
-----	-----------	-----	--------

uint	+ - * / ^	uint	-> uint
------	-----------	------	---------

float	+ - * / ^	float	-> float
-------	-----------	-------	----------

char	+ - * / ^	char	-> char
------	-----------	------	---------

Język jest silnie typowany, stąd w przypadku konieczności użycia operatora matematycznego na parze zmiennych o odmiennych typach konieczne jest użycie operatora konwersji `as`.

Warto pamiętać, że literały całkowitoliczbowe mogą być dopasowane przez analizator semantyczny, w zależności od potrzeby, do typu `int`, `uint` lub `float` (ale nie do `char`!), stąd jeśli jeden z operandów używanych do operacji operatorem arytmetycznym jest literałem całkowitoliczbowym, a drugi jest zmienną typu `int/uint/float`, to porównanie nie zgłasza błędu (za wyjątkiem sytuacji, gdy rzutowanie literału na określony typ skutkuje przekroczeniem zakresu).

Dzielenie liczb całkowitoliczbowych daje jako wynik, oczywiście, także liczbę całkowitoliczbową, i semantycznie odpowiada działaniu:

```
a / b -> (a as float / b as float) as int
```

To znaczy odrzuca część dziesiętną, którą otrzymalibyśmy dzieląc przez siebie te dwie liczby.

Jeśli na skutek działania operatora otrzymamy wynik nie mieszczący się w zakresie zmiennej wynikowej, interpreter zgłasza błąd, i z użyciem operacji modulo dokonuje zawinięcia, kontynuując próbę wykonania programu.

Przykłady błędów zgłaszanych przy działaniu operatorów arytmetycznych:

```
int x = 2'147'483'647
```

```
int y = x + 1
```

```
Error [Runtime] at line X, column Y, during operation: x + 1
```

```
Integer overflow: result does not fit into type int.
```

```
Value was wrapped using modulo arithmetic.
```

```
int x = 2'147'483'647 + 1
```

```
Error [Semantic] at line X, column Y, during operation: 2'147'483'647 + 1
```

```
Integer overflow: result does not fit into type int.
```

```
bool b = true
```

```
int x = 2 + b
```

```
Error [Semantic] at line X, column Y, during operation: 2 + b
```

```
Operator + not defined for [int] + [bool]
```

Różnica między błędem typu Runtime a semantycznym wynika stąd, że drugi spośród błędów używa literałów, z kolei pierwszy zmiennej `x`.

Operatory logiczne, operatory równości (i nierówności).

Operatory logiczne: Język `.djm` udostępnia użytkownikowi operatory logiczne `&&` (and) oraz `||` (or).

Są one zdefiniowane wyłącznie dla operandów typu `bool`:

```
bool && bool -> bool
```

```
bool || bool -> bool
```

```
bool b = true || 1
```

Error [Semantic] at line X, column Y, during operation: true || 1

Operator || not defined for [bool] || [integer literal]

Dopuszczalne jest użycie jako operandów literalów logicznych true i false. Priorytet operatora && jest wyższy niż priorytet operatora ||, stąd poniższe operacje są równoważne:

```
true || false && false
```

```
true || (false && false)
```

Operatory równości (i mniejszości):

W języku .djm dostępne są następujące:

operatory równości: !=, ==

operatory mniejszości: >=, >, <, <=

int	!= / == / < / > / <= / >=	int -> bool
uint	!= / == / < / > / <= / >=	uint -> bool
float	!= / == / < / > / <= / >=	float -> bool
bool	!= / == / < / > / <= / >=	bool -> bool
char	!= / == / < / > / <= / >=	char -> bool
string	!= / == / < / > / <= / >=	string -> bool

Dla typów int, uint, float, char semantyka powyższych operatorów jest oczywista.

Dla typów bool przyjmuje się, że false < true

Dla typów string, będących równoważnym list<char>, dokonywane jest po kolei porównywanie jak dla char. Jeśli krótszy wyraz jest początkiem dłuższego, ten krótszy jest traktowany jako mniejszy.

Operandami operatorów równości/mniejszości mogą być także listy

```
list<T>      != / == / < / > / <= / >=      list<T> -> bool
```

Jeśli listy są różnej długości, to porównywane są po kolei elementy krótszej do dłuższej, i jeśli to nie rozstrzyga o mniejszości (ponieważ krótsza lista jest początkiem dłuższej), to krótsza lista jest traktowana jako mniejsza. Obydwie porównywane listy muszą być tego samego typu.

Użycie operatorów mniejszości/większości dla klas zdefiniowanych przez użytkownika języka .djm zostały omówione w kolejnej sekcji, dotyczącej klas, i wymagają zdefiniowania dla klasy specjalnej funkcji compare i/lub equals.

Priorytety wszystkich operatorów, od najwyższego do najniższego:

^

*, /

+, -
<, <=, >, >=
==, !=
&&
||

Klasy w języku .djm

```
class User {  
    private name: string  
    private age: int  
    private isActive: bool  
    private country: string  
  
    static userCount: int = 0  
  
    User(name: string, age: int, country: string) {  
        this.name = name  
        this.age = age  
        this.country = country  
        this.isActive = true  
  
        User.userCount = User.userCount + 1  
    }  
  
    fun getName() -> string {  
        return this.name  
    }  
  
    mut fun setName(name: string) -> void {  
        this.name = name  
    }  
  
    fun getAge() -> int {  
        return this.age  
    }  
  
    mut fun setAge(age: int) -> void {  
        this.age = age  
    }  
  
    fun isAdult() -> bool {  
        return this.age >= 18  
    }  
  
    fun getCountry() -> string {  
        return this.country  
    }  
}
```

```

    }

    mut fun deactivate() -> void {
        this.isActive = false
    }

    static fun getUserCount() -> int {
        return User.userCount
    }

    fun equals(other: User) -> bool {
        return this.name == other.name && this.age == other.age && this.country ==
other.country
    }

    fun compare(other: User) -> int {
        return this.age - other.age
    }
}

mut User user = User("Ania", 25, "Polska")

```

Definiowanie klas najlepiej rozpatrzyć na przykładzie, ukazanym powyżej. Atrybuty klasy mogą zostać zadeklarowane ze słowem kluczowym `private`, które sprawia, iż do atrybutu którego dotyczy nie można się odwołać bezpośrednio, używając notacji z nazwą obiektu klasy i nazwy atrybutu rozdzielonych kropką (`nazwaObiektuKlasy.prywatny_atrybut`):

```

mut User user = User("Ania", 25, "Polska")
user.name = ...

```

Error [Semantic] at line X, column Y, during assignment: `user.name = ...`
Attribute name of class User is private

Zadeklarowanie atrybutu lub metody klasy z poprzedzającym deklarację słowem kluczem `static` powoduje, że dany atrybut lub metoda jest powiązana z klasą jako typem, a nie z konkretnym obiektem tej klasy. Próba wywołania takiej metody lub odwołanie się do atrybutu `static` z użyciem notacji z kropką skutkuje błędem semantycznym:

```

mut User user = User("Ania", 25, "Polska")
user.getUserCount()
user.userCount

```

Error [Semantic] at line X, column Y, during method call: `user.getUserCount()`
Static method `getUserCount` must be called using class name `User`

Error [Semantic] at line X, column Y, during member access: `user.userCount`
Static attribute `userCount` must be accessed using class name `User`

Warto zauważyć, że metoda static nie może używać słowa kluczowego this, a próba go użycia wewnątrz ciała takiej metody jest błędem semantycznym.

Konstruktor tworzy się z użyciem identyfikatora klasy. Możliwe jest utworzenie kilku konstruktorów, z różniącymi się argumentami - w takim przypadku podczas ich używania w kodzie programu analizator semantyczny decyduje, który z nich wywołać. Zdefiniowanie dwóch konstruktorów o takich samych argumentach (typach, liczności, kolejności) skutkuje błędem semantycznym.

W przypadku braku zdefiniowania konstruktora, przyjmowane są wartości domyślne dla typów prostych. Jeśli polem jest obiekt klasy, wartości domyślne przyjmowane są w głąb.

Metody mogą być oznaczone jako modyfikujące stan obiektu lub nie (zależnie od tego, czy dodane zostanie słowo kluczowe mut przed definicją danej metody klasy). Jeśli analizator semantyczny wykryje, że wewnątrz metody zdefiniowanej bez użycia mut próbujemy modyfikować stan obiektu poprzez przypisanie innej wartości do pola klasy, lub wywołanie metody oznaczonej jako mut na polu klasy będącym obiektem innej klasy, zgłaszany zostaje błąd semantyczny.

```
class User {  
    name: string  
  
    fun setName(name: string) -> void {  
        this.name = name  
    }  
}
```

Error [Semantic] at line X, column Y: setName
Cannot assign to field this.name inside non-mut method

```
class Profile {  
    mut fun activate() -> void { }  
}
```

```
class User {  
    profile: Profile  
  
    fun activateProfile() -> void {  
        this.profile.activate()  
    }  
}
```

Error [Semantic] at line X, column Y: activateProfile
Cannot call mut method activate on class User attribute profile of type Profile inside of non-mut method activateProfile

W ciele metody klasy analizator semantyczny w pierwszej kolejności przeszukuje lokalny zakres (scope) metody przy przeszukiwaniu tablicy symboli po napotkaniu identyfikatora. Jednoznaczne odwołanie się do atrybutu klasy wymaga więc użycia słowa kluczowego `this`.

```
class User {  
    name: string  
  
    mut fun setName(name: string) -> void { this.name = name }  
}
```

Statyczne typowanie i mutowalność

Statyczne typowanie języka .djm oznacza, że typ zmiennej jest podczas analizy semantycznej przypisywany do określonej zmiennej wewnątrz tablicy symboli, i jeśli analizator semantyczny wykryje próbę przypisania do zmiennej typu X literału (lub wartości innej zmiennej) typu Y, to zgłosi do modułu obsługi błędów błąd semantyczny.

```
mut int a = 5  
a = 7 // błąd semantyczny statyczny  
a = "tekst" // błąd typu (semantyczny statyczny)  
Error [Semantic] at line X, column Y, during assignment: a = "tekst"  
Cannot assign value of type string [StringLiteral] to variable of type int
```

Warto tutaj powtórzyć, że język jest silnie typowany, i co do zasady nie jest dokonywana niejawną konwersją przy próbie przypisania literału (lub innej zmiennej) do danej zmiennej. Jedynym wyjątkiem jest sytuacja, gdy próbujemy przypisać literał całkowitoliczbowy do zmiennej typu `int`/`uint`/`float` - wtedy zostanie dokonana niejawną konwersja (w przeciwieństwie do sytuacji, gdy chcielibyśmy taki literał traktować jednoznacznie jako typ `int` lub `uint`). Taka sytuacja **nie ma** miejsca, gdy próbujemy przypisać wartość innej zmiennej przez użycie jej identyfikatora.

```
int i = 5 // Poprawne przypisanie  
uint u = 5 // Poprawne przypisanie  
float f = 5 // Poprawne przypisanie
```

```
int a = 5  
uint b = a // błąd semantyczny  
Error [Semantic] at line X, column Y, during assignment: uint b = a  
Cannot assign value of variable of type int to variable of type uint  
float c = a // błąd semantyczny  
Error [Semantic] at line X, column Y, during assignment: float c = a  
Cannot assign value of variable of type int to variable of type float
```

Mutowalność w języku jest głęboka, co oznacza, że jeśli zdefiniujemy obiekt danej klasy A jako niemutowalny, to nie można nie tylko zmieniać wartości pól typu prostego niemutowalnego obiektu klasy A, ale w przypadku, gdy typem któregoś spośród pól klasy A jest inna klasa B, to nie można także na tym polu wywoływać metod `mut` ani przypisywać nowej wartości atrybutom obiektowi klasy B (będącego atrybutem klasy A).

```
mut User user = User("Ania", 25, "Polska")
```

```
u.name = "Kasia"
```

Error [Semantic] at line X, column Y, during assignment: u.name = "Kasia"

Object u is immutable

```
class Profile {
```

```
    login: string
```

```
    mut fun activate() -> void {}
```

```
}
```

```
class User {
```

```
    profile: Profile
```

```
}
```

```
User u = User()
```

```
u.profile.activate()
```

Error [Semantic] at line X, column Y, during method call: u.profile.activate()

Cannot call mut method activate on immutable object profile of class Profile

```
u.profile.login = "login"
```

Error [Semantic] at line X, column Y, during assignment: u.profile.login = "login"

Cannot assign value to attribute of immutable object profile of class Profile

Operatory list

Niżej opisane operatory list mogą działać w ogólności na różnych wyrażeniach, takich jak identyfikatory zmiennych, literały listowe czy wywołania funkcji lub metod zwracające listy.

Należy założyć, że przed zwróceniem wartości przez poniższe operatory dokonują jej (głębokiego, w przypadkach gdy zwracają listy zagnieżdżone lub obiekty klas) kopiowania.

Operator 1: ekstrakcja

Operacja ekstrakcji charakteryzuje się poniższą sygnaturą:

```
list<T>[int]
```

Służy do ekstrakcji elementu pod wskazanym indeksem. W przypadku list zagnieżdżonych możliwe jest, oczywiście, łańcuchowe użycie operatorów ekstrakcji.

Podanie do operatora ekstrakcji identyfikatora wyrażenia, które analizator semantyczny nie uzna za typ `int` skutkuje błędem semantycznym, natomiast podanie indeksu poza rozmiarem listy skutkuje błędem typu Runtime. Operator ekstrakcji nie akceptuje liczb ujemnych.

```
list<list<int>> matrix = [[1, 2], [3, 4, 5]]
```

```
int x = matrix[1][2] // 5
```

```
list<int> xs = [10, 20, 30]
```

```
int x = xs[true]
```

Error [Semantic] at line X, column Y, during list indexing: xs[true]

List index must have type int, got bool

```
list<int> xs = [10, 20, 30]
```

```
int x = xs[3]
```

Error [Runtime] at line X, column Y, during list indexing: xs[3]

Index 3 out of bounds for list of length 3

Dostępny jest operator wycinania zakresu xs[start:end], który zwraca podlistę w formie:

```
xs[start:end] = [xs[start], xs[start + 1], ..., xs[end - 1]]
```

Przy czym dopuszczalne są także skrócone formy:

```
xs[:end]    // od początku listy do end wyłącznie
```

```
xs[start:]  // od start do końca listy
```

Operator 2: konkatencja +

Operator + jest zdefiniowany (oprócz dla typów liczbowych) również dla typów listowych i ma sygnaturę:

```
list<T> + list<T> -> list<T>
```

Operator + zwraca nową listę i nie modyfikuje operandów.

```
list<int> a = [1, 2]
```

```
list<int> b = [3, 4]
```

```
list<int> c = a + b
```

```
c == [1, 2, 3, 4]
```

```
list<int> a = [1, 2]
```

```
list<int> b = a + 3
```

Error [Semantic] at line X, column Y, during operation: a + 3

Operator + not defined for list<int> + int

```
[1, 2] + ["a"]
```

Error [Semantic] at line X, column Y, during operation: [1, 2] + ["a"]

Operator + not defined for list<int> + list<string>

Operator 3: operator odejmowania -

Operator - jest zdefiniowany (oprócz dla typów liczbowych) również dla typów listowych i ma sygnaturę:

```
list<T> - list<T> -> list<T>
```

Operator - zwraca nową listę i nie modyfikuje operandów.

W ogólności semantyka dla odejmowania list może być opisana w sposób następujący:

1. Weź pierwszy element z prawego operandu
2. Następnie, zaczynając od początku listy będącej lewym operandem, porównuj po kolei z użyciem operatora `==` elementy lewej listy do elementu z punktu 1.
3. Jeśli operator `==` zwróci `true`, usuń element z lewej listy
4. Powtórz od kroku 1. dla kolejnego elementu z prawej listy

Działanie operatora - na listach zwraca, zgodnie z sygnaturą, typ listowy, i jest on łączny lewostronnie:

```
list<int> a = [1, 2, 3, 2]
```

```
list<int> b = [2, 5]
```

```
list<int> c = a - b           // [1, 3, 2]
```

```
list<int> d = a - b - [3]     // [1, 2]
```

```
(a - b) - [3] == a - b - [3]
```

Operator 3: operator przecięcia *

Operator `*` jest zdefiniowany (oprócz dla typów liczbowych) również dla typów listowych i ma sygnaturę:

```
list<T> * list<T> -> list<T>
```

Operator `+` zwraca nową listę i nie modyfikuje operandów.

W ogólności semantyka dla operatora przecięcia dla list może być opisana w sposób następujący:

Niech `a` - lewy operand, `b` - prawy operand.

1. Utwórz pomocniczą kopię listy `b` (oznaczymy ją jako `b_copy`).
2. Iteruj po elementach listy `a` od początku do końca.
3. Dla każdego elementu `x` z listy `a`:
 - a) przeszukaj listę `b_copy` od początku,
 - b) jeśli istnieje element `y` w `b_copy` taki, że `x == y`:
 - dodaj `x` do listy wynikowej,
 - usuń pierwsze dopasowanie `y` z `b_copy`,
 - c) jeśli takiego elementu nie ma, przejdź do kolejnego elementu `a`.
4. Zwróć listę wynikową.

Operator `*` jest, podobnie jak operator `+` łączny lewostronnie.

Intuicyjny opis, wynikające z wyżej opisanej semantyki, można przedstawić jako:

- element trafia do wyniku tyle razy, ile wynosi minimum liczby jego wystąpień w obu listach
- kolejność elementów w wyniku odpowiada kolejności ich występowania w lewej liście

```
[1, 2, 3] * ["jeden"]
```

Error [Semantic] at line X, column Y, during operation: [1, 2, 3] * ["jeden"]
Operator * not defined for list<int> + list<string>

Operator 4: operator grupowania %

Operator % służy do grupowania listy na podlisty według wartości wyrażenia obliczanego dla każdego spośród elementów listy podlegającej grupowaniu. Jego sygnaturę można więc w sposób ogólny opisać następująco:

```
list<T> % expr -> list<list<T>>
```

W wyrażeniu po prawej stronie operatora grupowania % dostępne jest słowo kluczowe `this`. Dla `list<T>`, `this` zwróci typ `T`. W poniższym więc przykładzie:

```
list<int> xs = [1, 2, 1, 3, 2]  
list<list<int>> grouped = xs % this
```

otrzymamy:

```
[  
  [1, 1],  
  [2, 2],  
  [3]  
]
```

Oczywiście `this` można użyć do otrzymania wartości atrybutu lub metody, jeśli dokonujemy operacji grupowania na liście przechowującej obiekty klas.

```
class Group {  
  key: string  
  value: int  
  isActive: bool
```

```
Group(key: string, value: int, isActive: bool) {  
  this.key = key  
  this.value = value  
  this.isActive = isActive  
}  
}
```

```
list<Group> groups = [  
  Group("A", 10, true), Group("B", 20, false),  
  Group("A", 30, true), Group("C", 40, true),  
  Group("B", 50, true)]
```

```
list<list<Group>> grouped = groups % this.key
```



```
[
  [ Group("A", 10, true),   Group("A", 30, true) ],
  [ Group("B", 20, false),  Group("B", 50, true) ],
  [ Group("C", 40, true) ]
]
```

Operator grupowania % używa w celu porównywania kolejnych wartości wyrażenia prawego operandu operatora ==, stąd musi on być zdefiniowany dla typu przechowywanego przez listę, która podlega grupowaniu (tzn. jest lewym operandem operatora %).

Operator 5: operator filtrowania ?

Operator ? służy do filtrowania listy na podstawie wartości pewnego wyrażenia logicznego. Sygnaturę można opisać następująco:

```
list<T> ? expr -> list<T>
```

Przy czym wyrażenie expr zwraca wartość boolowską. Analogicznie, jak w przypadku wcześniej opisanego operatora grupowania %, wewnątrz wyrażenia expr można użyć słowa kluczowego this, umożliwiającego dostęp do przetwarzanego elementu listy, na której dokonywane jest filtrowanie. Działanie operatora filtrowania ? wyjaśnia poniższy przykład:

```
list<int> xs = [1, 2, 3, 4, 5]
list<int> greaterThanTwo = xs ? (this > 2)
```

Operator filtrowania nie modyfikuje listy będącej jego lewym operandem, stąd w poniższym przykładzie było konieczne zastosowanie operatora przypisania w celu zmiany wartości zmiennej xs typu list<int>. Weryfikacja typu zwracanego przez wyrażenie będącego lewym operandem ? następuje na etapie analizy semantycznej, i interpreter .djm zgłasza błąd semantyczny gdy zwracana wartość jest typu innego niż bool.

Za pomocą this można także uzyskać dostęp do atrybutów i metod obiektów klas.

```
class User {
  name: string
  age: int
  isActive: bool

  User(name: string, age: int, isActive: bool) {
    this.name = name
    this.age = age
    this.isActive = isActive
  }
}
```

```
list<User> users = ...
list<User> activeAdults = users ? (this.age > 18 && this.isActive)
```

Operator 6: operator mapowania |>

Operator mapowania dokonuje rzutowania elementów listy na wyrażenie będące jego prawym operandem, i zwraca listę, której elementy są po kolei efektem tego rzutowania. Jego sygnaturę można opisać poprzez:

```
list<T> |> expr -> list<U>
```

Wyrażenie `expr` może zwracać typ `U` inny niż typ `T` przechowywany przez listę na której dokonywana operacja mapowania.

Wewnątrz wyrażenia `expr`, będącego prawym operandem, można (podobnie jak w przypadku wcześniej opisanych operatorów grupowania i filtrowania) używać słowa kluczowego `this`, dającego dostęp do obecnie przetwarzanego elementu listy.

```
fun toThePowerOf(int: x, int: y) -> int {  
    return x^y  
}
```

```
list<int> xs = [1, 2, 3]  
list<int> xs_squared = xs |> toThePowerOf(this, 2)  
xs_squared == [1, 4, 9] // true
```

Operator 7: operator (unarny) liczności count

Operator unarny `count` zwraca długość listy. Ma on sygnaturę:

```
count list<T> -> uint
```

```
count (users ? isActive) // Zwraca liczbę aktywnych użytkowników
```

Operator 8: operator (unarny) odwrócenia reverse

Operator unarny `reverse` zwraca listę odwróconą. Nie modyfikuje listy na której jest używany. Ma on sygnaturę:

```
reverse list<T> -> list<T>
```

Operator 9: operator (unarny) spłaszczenia flatten

Operator `flatten` spłaszcza listę o jeden poziom. Nie modyfikuje listy wejściowej. Ma następującą sygnaturę:

```
flatten list<T> -> list<T>
```

Jego semantykę można opisać w sposób następujący:

Niech `xs` będzie listą typu `list<list<T>>`.

1. Utwórz pustą listę wynikową typu `list<T>`.
2. Iteruj po elementach listy `xs` od lewej do prawej.
3. Dla każdego elementu `x`:

- a) `x` ma typ `list<T>`,
 - b) iteruj po elementach listy `x` od lewej do prawej,
 - c) dodawaj kolejne elementy do listy wynikowej.
4. Zwróć listę wynikową.

```
list<list<int>> xs = [[1, 2], [3, 4], [5]]
list<int> ys = flatten xs
```

```
ys == [1, 2, 3, 4, 5] // true
```

Można go użyć jedynie dla list zagnieżdżonych:

```
list<int> xs = [1, 2, 3]
flatten xs
```

Error [Semantic] at line X, column Y, during operation: flatten xs
Operator flatten is defined only for list<list<T>>, got list<int>

Operator 10: operator zawierania contains

Operator zawierania contains ma następującą sygnaturę:

```
list<T> contains list<T> -> bool
```

Zwraca on true jedynie wtedy, kiedy każdy spośród elementów z prawego operandu jest zawarty w lewym. Taka semantyka została przyjęta, gdyż w celu sprawdzenia, czy zbiory wartości dwóch list w ogóle się przecinają, można użyć krótkiego zapisu złożonego z operatora unarnego liczności count oraz operatora przecięcia list *, jak poniżej:

```
count (list1 * list2) > 0
```

A podobny zapis w celu sprawdzenia, czy jedna lista jest w całości zawarta jest w drugiej wymagałby porównania otrzymanego przez count wyniku z długością krótszej z list, co z kolei wymagałoby także wcześniejszej wiedzy o tym jaka ta długość jest. Byłby więc dłuższy, stąd tak przyjęta semantyka operatora contains, który w założeniu miał zastąpić dłuższą z operacji.

Definiowanie funkcji

Funkcje w języku .djm definiuje się poniższym schematem:

```
fun nazwa(param1: Typ1, param2: Typ2) -> TypWyniku {
  ...
}
```

Wszelkie argumenty są przekazywane przez wartość, co oznacza także konieczność (w przypadku list zagnieżdżonych oraz klas) głębokiego kopiowania, które dla obiektów klas stworzonych przez użytkownika zostało szerzej opisane w poprzedniej części dokumentacji, w sekcji dotyczącej klas.

```

class User {
    name: string

    User(name: string) {
        this.name = name
    }

    mut fun setName(name: string) -> void {
        this.name = name
    }
}

fun rename(u: User) -> User {
    u.setName("Anna")
    return u
}

User user = User("Kasia")
User renamed = rename(user)

user.name == "Kasia" // true
renamed.name == "Anna" // true

```

Jako typ zwracany przez funkcję dopuszczalne jest użycie słowa kluczowego `void`, niemniej jednak w każdym innym przypadku obowiązkowe jest użycie w ciele funkcji `return` wraz z wyrażeniem (identyfikatorem, literałem, wyrażeniem złożonym) dający się zinterpretować jako pasujący typ.

Z racji, że argumenty są przekazywane przez wartość, nie zostało uznane za konieczne zapisywanie `mut` przed identyfikatorem wewnątrz listy argumentów przy definicji funkcji, i wewnątrz ciała funkcji wszelkie przekazane argumenty są traktowane jako mutowalne. Stąd w powyższym przykładzie, wewnątrz ciała funkcji `rename` dopuszczalne jest użycie metody `setName` na obiekcie klasy `User` przekazywanym jako argument z identyfikatorem `u`.

Jeśli chcemy, żeby funkcja modyfikowała stan obiektu, to sugerowane jest dopisanie w ciele definicji klasy odpowiedniej metody lub, jak w przykładzie powyżej, użycie konstrukcji:

```

User renamed = rename(user)
T t = functionModifyingAndReturningT(another_t)

```

Ta decyzja projektowa została podjęta w celu minimalizacji poziomu skomplikowania języka i ułatwienia zaimplementowania jego interpretera.

Pętla for

```
for T t in expression {  
    ...  
}
```

Jako expression w powyższym schemacie należy podać wyrażenie zwracające listę złożoną z elementów typu T (czyli list<T>). Może to być zarówno literał, jak i identyfikator czy wyrażenie złożone, używające przykładowo jednego z dostępnych dla list operatorów.

```
for User u in users ? this.isActive {  
    print(u.name)  
}
```

Konieczność zapisania typu T wymaga na programiście języka .djm świadomości zwracanego przez expression typu oraz zgodna jest z ideą statycznego typowania, którym charakteryzuje się język .djm

Jeśli decydujemy się przekazać do pętli for (jako expression w powyższym przykładzie) sam identyfikator zmiennej listowej, to możliwość zmiany tej listy i jej składowych wewnątrz pętli for jest warunkowana tym, czy przekazywana zmienna listowa została zadeklarowana jako mutowalna czy niemutowalna.

```
list<User> users = [User( ... ), User( ... ), ... ] // lista niemutowalna  
for User u in users ? this.isActive {  
    u.name = "newName" // Błąd semantyczny, Error [Semantic] ...  
}
```

Instrukcja if

W języku .djm instrukcja if ma standardową, znaną z wielu innych języków programowania postać.

```
if bool_expr {  
    //  
} bool_expr {  
    //  
} else {  
    //  
}
```

Jeśli jako bool_expr podane zostanie wyrażenie które nie zwraca wartości boolowskiej (w tym identyfikator zmiennej typu innego niż bool, jako że w języku właściwie nie występują niejawne konwersje) zostanie zgłoszony jako błąd semantyczny.

Instrukcja while

```
while bool_expr {  
    //  
}
```

Podobnie jak w przypadku instrukcji if, jeśli jako `bool_expr` podane zostanie wyrażenie które nie zwraca wartości boolowskiej (w tym identyfikator zmiennej typu innego niż `bool`, jako że w języku właściwie nie występują niejawne konwersje) zostanie zgłoszony błąd semantyczny.

Zakresy widoczności i przesłanianie (shadowing) zmiennych

Język .djm wykorzystuje lokalne zakresy widoczności zmiennych (pomiędzy nawiasami klamrowymi `{ }`).

Wyszukiwanie identyfikatorów odbywa się od obecnego (najbliższego) zakresu do najbardziej zewnętrznego, stąd możliwe jest redefiniowanie zmiennych w wewnętrznych zakresach o identyfikatorach takich samych, jak te z wyższych zakresów. Zachodzi wtedy ich przykrywanie.

```
mut int x = 5  
  
{  
    mut int x = 2  
    {  
        x == 2    // true  
    }  
}
```

Jako że nawiasy klamrowe są używane przy zapisywaniu ciał funkcji, pętli, instrukcji if i while oraz ciał klas, to efekt przykrywania zmiennych widoczny jest także w tych przypadkach.

Importowanie plików źródłowych i zmienne (stałe) globalne

W języku .djm udostępniono mechanizm importowania plików źródłowych. Importowania można dokonać zarówno poprzez użycie tzw. wildcard (oznaczonego gwiazdką), co powoduje zaimportowanie wszystkich identyfikatorów (funkcji, klas, zmiennych globalnych) z danego pliku, jak i możliwość importowania jedynie wybranych elementów. Importowanie jest dokonywane za pomocą następujących instrukcji:

```
import * from "file1.djm"  
import Class2, fun2, var2 from "../dir/to/file2.djm"
```

Które muszą znaleźć się na początku pliku źródłowego .djm - wymagane to jest przez samą gramatykę języka i ma na celu wyraźne zaznaczenie importów na samym początku.

Niezależnie od rodzaju (z użyciem gwiazdki, czy selektywny) importu jaki zostanie zastosowany, importowanie wprowadza nową przestrzeń nazw, będącą nazwą pliku bez rozszerzenia ".djm". Stąd odwołanie się w pliku źródłowym do zaimportowanych w powyższym przykładzie identyfikatorów musi odbywać się w następujący sposób:

```
file1.fun1(), file2.fun2()
file1.var1, file2.var2,
file2.Class2 class2Obj = file2.Class2(...)
```

W języku można używać zmiennych globalnych, jednak muszą one być zadeklarowane (i najlepiej od razu zdefiniowane, gdyż w przeciwnym wypadku będą miały na stałe domyślną wartość) bez użycia słowa kluczowego `mut`, co w istocie czyni je stałymi. Bardziej niż zmienne globalne, można by więc określić je mianem globalnych stałych.

Sposób uruchomienia interpretera i funkcja main

Interpreter uruchamiany jest z poziomu terminala. Jako argument należy podać w pierwszej kolejności ścieżkę do (głównego) pliku źródłowego .djm, zaś jako kolejne mogą zostać podane (opcjonalne) argumenty funkcji main.

```
djm-interpreter ./main.djm
```

Jako że funkcja main stanowi punkt wejścia programu, musi być ona zdefiniowana w pliku który podamy jako główny (tzn. jako pierwszy argument przy wywołaniu interpretera z terminala). Jeśli któreś spośród importowanych do głównego pliku źródłowego pliki .djm mają wewnątrz zdefiniowaną funkcję main, zostaje ona zignorowana podczas dokonywania importowania.

Funkcja main ma następującą sygnaturę:

```
fun main(args: list<string>) -> int {
    return 0
}
```

Wszelkie argumenty podane podczas wywoływania interpretera djm z konsoli są zapisywane jako typ string, w związku z czym ich ewentualne użycie w charakterze innych typów wymaga jawnej konwersji przez programistę języka djm.

Analizator leksykalny i rozpoznawane tokeny

Poniżej wypisane zostały wszystkie spośród rozpoznawanych przez lekser tokenów

Literały:

```
INT_LITERAL, FLOAT_LITERAL, STRING_LITERAL, CHAR_LITERAL,
BOOL_LITERAL
```

Dokładny opis literałów znajduje się na początku tego dokumentu, w sekcji Literały.

Słowa kluczowe:

**KW_INT, KW_UINT, KW_FLOAT, KW_BOOL, KW_CHAR, KW_STRING,
KW_LIST, KW_VOID**

**KW_CLASS, KW_FUN, KW_RETURN
KW_MUT, KW_PRIVATE, KW_STATIC
KW_IF, KW_ELSE, KW_WHILE, KW_FOR, KW_IN
KW_IMPORT, KW_FROM
KW_THIS**

Odpowiadają one słowom kluczowym

`int, uint, float, ..., mut, private, ..., import, from, this`

Operatory:

**+ - * / ^
== != <= >= < >
&& || !
=
->
|> ? %
CONTAINS AS COUNT REVERSE FLATTEN**

Symbole:

() [] { } , . : _

Identyfikatory:

Muszą zaczynać się od litery, oraz mogą składać się z liter (małych i dużych), cyfr oraz podkreślników.

Przykładowy rozkład kawałka kodu na tokeny przez lekser znajduje się poniżej:

`xs contains ys_02[2 : count ys_02]`

```
IDENTIFIER(xs)
contains           // czyli OP_CONTAINS
IDENTIFIER(ys_02)
[                 // czyli LBRACKET
INT_LITERAL(2)
:                 // czyli SYMBOL_COLON
count            // czyli OP_COUNT
IDENTIFIER(ys_02)
]                 // czyli RBRACKET
```

W przykładzie wypisano proponowane nazwy, jakie mogą zostać w końcowym projekcie użyte wewnątrz enum w którym wymienione zostaną tokeny możliwe do rozpoznania przez lekser.

Jako że separatorami w literałach liczbowych są podkreślniki, a identyfikatory zmiennych, mimo że mogą składać się z liter/cyfr/podkreślników, muszą zaczynać się od litery (a więc muszą zawierać co najmniej jedną literę) możliwe jest już na etapie leksera rozdzielanie identyfikatorów od literałów liczbowych.

Lekser, podczas tworzenia tokenów, będzie zapisywał w strukturze danych przechowujących tokeny ich linię, kolumnę, oraz plik z którego pochodzą. Takie dane umożliwią modułowi obsługi błędów wypisanie fragmentu kodu, który je spowodował, bez względu na warstwę analizy, podczas której one wystąpią.

Parser (analizator składniowy)

Parser komunikuje się z modulem leksera, wnioskując o utworzenie i przekazanie mu kolejnego tokenu. Powinien mieć on możliwość podejrzenia następnego tokenu, pobrania obecnego, przejścia dalej, ale przede wszystkim także sprawdzenia typu tokenu (dopuszczalne typy tokenów zostały wypisane powyżej, w sekcji Analizator leksykalny).

Mogąc dokonać podczas komunikacji z modulem leksera takich operacji, parser ma możliwość na bieżąco sprawdzać poprawność kodu dlm z regułami gramatyki języka. Gdy napotka na fragment kodu, którego nie da się dopasować do żadnej z reguł gramatycznych, może od razu zgłosić do modułu obsługi błędów błąd składniowy.

Zakładając, że kod jest poprawny, parser będzie mógł, z użyciem reguł gramatyki złożyć otrzymane tokeny w większe, wielotokenowe struktury. Prosty przykład znajduje się poniżej:

`xs[2:4]`

```
SliceExpression  
____object: Identifier(xs)  
____start: IntLiteral(2)  
____end: IntLiteral(4)
```

UWAGA: Poniższy fragment jest jedynie przykładem, ukazującym pewną ideę, bez gwarancji, że dokładnie tego typu struktura znajdzie się rzeczywiście w kodzie C++ interpretera dlm.

Analizator semantyczny i tablica symboli

Analizator semantyczny otrzymuje od parsera drzewko AST, zawierające struktury wynikające z reguł gramatyki języka. Przechodzi on po otrzymanym drzewku AST, i dokonuje analizy semantycznej, tzn.:

- Sprawdza czy zmienne zostały zadeklarowane przed użyciem, czy nie ma niedozwolonych redefinicji.
- Czy typy
(
zarówno, gdy są to identyfikatory zmiennych, których typ można sprawdzić

w tablicy symboli, czy są to wywołania funkcji, przez które zwracane typy również można zweryfikować w tablicy symboli, jak i gdy są to literały, których typ został określony przez analizator leksykalny, a następnie przekazany przez parser)

używane jako operandy, argumenty funkcji, przypisywane do zmiennych operatorem przypisania (i inne działania) są zgodne z tymi dopuszczalnymi przez definicję operatorów i funkcji

- Czy w instrukcji for ... in ... po słowie kluczowym in rzeczywiście użyto wyrażenia listowego, czy w instrukcjach if oraz while użyto wyrażeń boolowskich

i wielu innych, dla których przykładowe błędy semantyczne wymieniono we wcześniejszej części tego dokumentu.

Tablica symboli jest tworzona właśnie na etapie analizy semantycznej. Zawiera ona identyfikatory oraz typy zmiennych, a także identyfikatory funkcji wraz z typami przez nie zwracanymi oraz typami argumentów, jakie te funkcje przyjmują.

W związku z opisanym w sekcji **Zakresy widoczności...** przesłanianiem zmiennych, konieczna jest hierarchiczna struktura tablicy symboli.

Co konkretnie będzie zawierał pojedynczy wpis w tablicy symboli zależne jest od rodzaju tego wpisu. Poniżej wyszczególniono wstępny ich zarys:

Zmienna:

___ nazwa
___ typ
___ czy mutowalna

Funkcja:

___ nazwa
___ lista parametrów
___ nazwa
___ typ
___ typ zwracany

Klasa:

___ nazwa
___ pola klasy
___ czy zadeklarowana jako static (patrz sekcja **Klasy w języku .djm**)
___ metody klasy
___ czy zadeklarowana z przedrostkiem mut (patrz sekcja **Klasy w języku .djm**)
___ czy zadeklarowana jako static (patrz sekcja **Klasy w języku .djm**)
___ konstruktory

Istotne jest umieszczenie w tablicy symboli informacji o tym, czy dane metody klasy są mut lub static, gdyż umożliwi to wykrycie i zgłoszenie (do modułu obsługi błędów) błędów semantycznych szerzej opisanych w sekcji **Klasy w języku .djm**.

Gramatyka EBNF

//

```
digit_sequence = digit , { digit | "_" , digit } ;
int_literal    = digit_sequence ;
float_literal   = digit_sequence , "." , digit_sequence ;
string_literal  = "\"" , { string_char } , "\"" ;
list_literal    = "[" , [ expression , { "," , expression } ] , "]" ;
char_literal    = "'" , char_char , "'" ;
bool_literal    = "true" | "false" ;
```

```
literal        = float_literal
                | int_literal
                | string_literal
                | char_literal
                | bool_literal
                | list_literal ;
```

```
identifier     = letter , { letter | digit | "_" } ;
identifier_list = identifier , { "," , identifier } ;
```

//

```
type           = value_type
                | "void" ;
```

```
value_type     = basic_value_type
                | list_type
                | user_type ;
basic_value_type
    = "int"
    | "uint"
    | "float"
    | "bool"
    | "char"
    | "string" ;
```

```
list_type      = "list" , "<" , value_type , ">" ;
user_type      = identifier ;
```

//

```
program        = { import_decl } , { top_level_decl } ;
```

```
top_level_decl = class_decl
                | function_decl
                | global_const_decl ;
```

```
global_const_decl = value_type , identifier_list , [ "=", expression ] ;  
variable_decl_stmt = [ "mut" ] , value_type , identifier_list , [ "=", expression ] ;
```

```
block = "{" , { statement } , "}" ;
```

```
statement = variable_decl_stmt  
| assignment_stmt  
| expression  
| return_stmt  
| if_stmt  
| while_stmt  
| for_stmt  
| block ;
```

```
////////////////////////////////////////////////////////////////
```

```
import_spec = "*"   
| identifier , { "," , identifier } ;
```

```
import_decl = "import" , import_spec , "from" , string_literal ;
```

```
////////////////////////////////////////////////////////////////
```

```
parameter_list = parameter , { "," , parameter } ;
```

```
parameter = identifier , ":", value_type ;
```

```
function_decl = "fun" , identifier ,  
"(" , [ parameter_list ] , ")" ,  
"->" , type ,  
block ;
```

```
////////////////////////////////////////////////////////////////
```

```
class_decl = "class" , identifier , "{" , { class_member } , "}" ;
```

```
constructor_decl  
= identifier ,  
"(" , [ parameter_list ] , ")" ,  
block ;
```

```
class_member = field_decl  
| method_decl  
| constructor_decl ;
```

```
field_modifier = "private"  
| "static" ;
```

```
field_decl    = [ field_modifier ] ,  
               identifier_list , ":" , value_type ,  
               [ "=" , expression ] ;
```

```
method_modifier = "mut"  
                 | "static" ;
```

```
method_decl    = [ method_modifier ] ,  
                 function_decl ;
```

```
////////////////////////////////////////////////////////////////
```

```
assignment_postfix = "." , identifier | index_suffix ;  
assignment_target  = identifier , { assignment_postfix } ;  
assignment_stmt    = assignment_target , "=" , expression ;
```

```
return_stmt      = "return" , [ expression ] ;
```

```
if_stmt          = "if" , expression , block ,  
                  { "else" , "if" , expression , block } ,  
                  [ "else" , block ] ;
```

```
while_stmt       = "while" , expression , block ;
```

```
for_stmt         = "for" , value_type , identifier , "in" , expression , block ;
```

```
////////////////////////////////////////////////////////////////
```

```
expression = logical_or_expr ;
```

```
logical_or_expr = logical_and_expr ,  
                 { "|" , logical_and_expr } ;
```

```
logical_and_expr  
    = equality_expr ,  
      { "&&" , equality_expr } ;
```

```
equality_expr = relational_expr ,  
               [ ( "==" | "!=" | "contains" ) , relational_expr ] ;
```

```
relational_expr = additive_expr ,  
                 [ ( "<" | "<=" | ">" | ">=" ) , additive_expr ] ;
```

```
additive_expr = multiplicative_expr ,  
               { ( "+" | "-" ) , multiplicative_expr } ;
```

```
multiplicative_expr
```

```

    = list_operator_expr ,
      { ( "*" | "/" ) , list_operator_expr } ;

list_operator_expr
  = map_expr ;

map_expr      = filter_expr ,
               { ">" , filter_expr } ;

filter_expr   = group_expr ,
               { "?" , group_expr } ;

group_expr    = cast_expr ,
               { "%" , cast_expr } ;

cast_expr     = unary_expr ,
               { "as" , value_type } ;

unary_expr    = unary_op , unary_expr
               | power_expr ;

power_expr    = postfix_expr ,
               [ "^" , unary_expr ] ;

unary_op      = "-"
               | "!"
               | "count"
               | "reverse"
               | "flatten" ;

primary_expr  = literal
               | identifier
               | "(" , expression , ")"
               | "this" ;

////////////////////////////////////////////////////////////////

argument_list = expression , { "," , expression } ;
call_suffix   = "(" , [ argument_list ] , ")" ;
index_suffix  = "[" , expression , "]" ;
slice_suffix  = "[" , [ expression ] , ":" , [ expression ] , "]" ;

postfix_expr  = primary_expr , { postfix_op } ;
postfix_op    = "." , identifier , [ call_suffix
               | index_suffix
               | slice_suffix
               | call_suffix ;

```

////////////////////////////////////